Software Re-Engineering

Lecture 18

Reverse engineering III – IDA Data structures

Instructor:

Dr. Natalia Chaudhry,

Assistant Professor, Software Engineering Department, PU.

SE Fall 2020

Readings

Chapter 8: DATA TYPES AND DATA STRUCTURES, The IDA Pro Book, 2nd Edition: The Unofficial Guide to the World's Most Popular Disassembler Second Edition by Chris Eagle

Recognizing Data Structure Usage

Array Member Access

Globally Allocated Arrays

```
int global_array[3];

int main() {
   int idx = 2;
   global_array[0] = 10;
   global_array[1] = 20;
   global_array[2] = 30;
   global_array[idx] = 40;
}
```

```
.text:00401000 _main            proc near
.text:00401000
.text:00401000 idx              = dword ptr -4
.text:00401000
.text:00401000                  push    ebp
.text:00401001                  mov     ebp, esp
.text:00401003                  push    ecx
.text:00401004                  mov     [ebp+idx], 2
.text:0040100B                ❶mov      dword_40B720, 10
.text:00401015                ❷mov      dword_40B724, 20
.text:0040101F                ❸mov      dword_40B728, 30
.text:00401029                  mov     eax, [ebp+idx]
.text:0040102C                ❹mov      dword_40B720[eax*4], 40
.text:00401037                  xor     eax, eax
.text:00401039                  mov     esp, ebp
.text:0040103B                  pop     ebp
.text:0040103C                  retn
.text:0040103C _main            endp
```

Recognizing Data Structure Usage

Array Member Access

While this program has only one global variable, the disassembly lines at **1, 2,** and **3** seem to indicate that there are three global variables.

The computation of an offset (eax * 4) at **4** is the only thing that seems to hint at the presence of a global array named dword_40B720, yet this is the same name as the global variable found at .

```
.text:00401000 _main            proc near
.text:00401000
.text:00401000 idx              = dword ptr -4
.text:00401000
.text:00401000                  push    ebp
.text:00401001                  mov     ebp, esp
.text:00401003                  push    ecx
.text:00401004                  mov     [ebp+idx], 2
.text:0040100B                ❶mov      dword_40B720, 10
.text:00401015                ❷mov      dword_40B724, 20
.text:0040101F                ❸mov      dword_40B728, 30
.text:00401029                  mov     eax, [ebp+idx]
.text:0040102C                ❹mov      dword_40B720[eax*4], 40
.text:00401037                  xor     eax, eax
.text:00401039                  mov     esp, ebp
.text:0040103B                  pop     ebp
.text:0040103C                  retn
.text:0040103C _main            endp
```

Recognizing Data Structure Usage

Array Member Access

Stack Allocated Arrays

```
int main() {
    int stack_array[3];
    int idx = 2;
    stack_array[0] = 10;
    stack_array[1] = 20;
    stack_array[2] = 30;
    stack_array[idx] = 40;
}
```

The address at which stack_array will be allocated is unknown at compile time, so it is not possible for the compiler to precompute the address of stack_array[1] at compile time as it did in the global array example

```
.text:00401000 _main            proc near
.text:00401000
.text:00401000 var_10           = dword ptr -10h
.text:00401000 var_C            = dword ptr -0Ch
.text:00401000 var_8            = dword ptr -8
.text:00401000 idx              = dword ptr -4
.text:00401000
.text:00401000                  push    ebp
.text:00401001                  mov     ebp, esp
.text:00401003                  sub     esp, 10h
.text:00401006                  mov     [ebp+idx], 2
.text:0040100D                ❶mov      [ebp+var_10], 10
.text:00401014                ❷mov      [ebp+var_C], 20
.text:0040101B                ❸mov      [ebp+var_8], 30
.text:00401022                  mov     eax, [ebp+idx]
.text:00401025                ❹mov      [ebp+eax*4+var_10], 40
.text:0040102D                  xor     eax, eax
.text:0040102F                  mov     esp, ebp
.text:00401031                  pop     ebp
.text:00401032                  retn
.text:00401032 _main            endp
```

As with the global array example, this function appears to have three variables (**var_10, var_C, and var_8**) rather than an array of three integers. Based on the constant operands used at **1, 2,** and **3**, we know that what appear to be local variable references are actually references to the three elements of stack_array whose first element must reside at **var_10**, the local variable with the lowest memory address.

```
.text:0040100D    ❶mov    [ebp+var_10], 10
.text:00401014    ❷mov    [ebp+var_C], 20
.text:0040101B    ❸mov    [ebp+var_8], 30
.text:00401022      mov    eax, [ebp+idx]
.text:00401025    ❹mov    [ebp+eax*4+var_10], 40
```

- consider what the compiler goes through when dealing with the reference to stack_array[1], which lies 4 bytes into the array, or 4 bytes beyond the location of var_10.
- Within the stack frame, the compiler has elected to allocate stack_array at ebp - 0x10. The compiler understands that stack_array[1] lies at ebp - 0x10 + 4, which simplifies to ebp - 0x0C. The resultis that IDA displays this as a local variable reference.

- The net effect is that, similar to globally allocated arrays, the use of constant index values tends to hide the presence of a stack-allocated array.
- Only the array access at **4** hints at the fact that var_10 is the first element in the array rather than a simple integer variable. In addition, the disassembly line at also helps us conclude that the size of individual elements in the array is 4 bytes.

```
.text:0040100D    ❶mov    [ebp+var_10], 10
.text:00401014    ❷mov    [ebp+var_C], 20
.text:0040101B    ❸mov    [ebp+var_8], 30
.text:00401022      mov    eax, [ebp+idx]
.text:00401025    ❹mov    [ebp+eax*4+var_10], 40
```

Stack-allocated arrays and globally allocated arrays are thus treated very similarly by compilers.

Recognizing Data Structure Usage

Array Member Access

Heap Allocated Arrays

```
int main() {
    int *heap_array = (int*)malloc(3 * sizeof(int));
    int idx = 2;
    heap_array[0] = 10;
    heap_array[1] = 20;
    heap_array[2] = 30;
    heap_array[idx] = 40;
}
```

Recognizing Data Structure Usage

Array Member Access

Heap Allocated Arrays

```
.text:00401000 _main      proc near
.text:00401000
.text:00401000 heap_array      = dword ptr -8
.text:00401000 idx             = dword ptr -4
.text:00401000
.text:00401000            push    ebp
.text:00401001            mov     ebp, esp
.text:00401003            sub     esp, 8
.text:00401006          ❺push     0Ch             ; size_t
.text:00401008            call    _malloc
.text:0040100D            add     esp, 4
.text:00401010            mov     [ebp+heap_array], eax
.text:00401013            mov     [ebp+idx], 2
.text:0040101A            mov     eax, [ebp+heap_array]
.text:0040101D          ❶mov      dword ptr [eax], 10
.text:00401023            mov     ecx, [ebp+heap_array]
.text:00401026          ❷mov      dword ptr [ecx+4], 20
.text:0040102D            mov     edx, [ebp+heap_array]
.text:00401030          ❸mov      dword ptr [edx+8], 30
```

```
.text:00401037            mov     eax, [ebp+idx]
.text:0040103A            mov     ecx, [ebp+heap_array]
.text:0040103D          ❹mov      dword ptr [ecx+eax*4], 40
.text:00401044            xor     eax, eax
.text:00401046            mov     esp, ebp
.text:00401048            pop     ebp
.text:00401049            retn
.text:00401049 _main      endp
```

The starting address of the array (returned from malloc in the EAX register) is stored in the local variable heap_array

- The references to **heap_array[0], heap_array[1],** and **heap_array[2]** require offsets of 0, 4, and 8 bytes, respectively, as seen at **1, 2,** and **3**.
- The operation that most closely resembles the previous examples is the reference to heap_array[idx] at **4**, in which the offset into the array continues to be computed by multiplying the array index by the size of an array element.

```
.text:0040101D    ❶mov    dword ptr [eax], 10
.text:00401023      mov    ecx, [ebp+heap_array]
.text:00401026    ❷mov    dword ptr [ecx+4], 20
.text:0040102D      mov    edx, [ebp+heap_array]
.text:00401030    ❸mov    dword ptr [edx+8], 30

.text:00401037      mov    eax, [ebp+idx]
.text:0040103A      mov    ecx, [ebp+heap_array]
.text:0040103D    ❹mov    dword ptr [ecx+eax*4], 40
```

```
.text:00401006          ❺push    0Ch             ; size_t
.text:00401008            call    _malloc
.text:0040100D            add     esp, 4
.text:00401010            mov     [ebp+heap_array], eax
.text:00401013            mov     [ebp+idx], 2
.text:0040101A            mov     eax, [ebp+heap_array]
.text:0040101D          ❶mov      dword ptr [eax], 10
.text:00401023            mov     ecx, [ebp+heap_array]
.text:00401026          ❷mov      dword ptr [ecx+4], 20
.text:0040102D            mov     edx, [ebp+heap_array]
.text:00401030          ❸mov      dword ptr [edx+8], 30
```

C / 4 (in hex) = 3 bytes

- For heap allocated arrays, the parameter passed to the memory allocation function (0x0C passed to malloc at **5**) represents the total number of bytes allocated to the array.
- Dividing this by the size of an element (4 bytes in this example, as observed from the offsets at **1, 2,** and **3**) tells us the number of elements in the array. In the previous example, a three-element array was allocated.

To Do

Study **Structure Member Access** and **Stack-Allocated Structures**

```
struct ch8_struct {    //Size     Minimum offset     Default offset
   int field1;         //  4              0                  0
   short field2;       //  2              4                  4
   char field3;        //  1              6                  6
   int field4;         //  4              7                  8
   double field5;      //  8              11                 16
};                  //Minimum total size: 19   Default size: 24
```

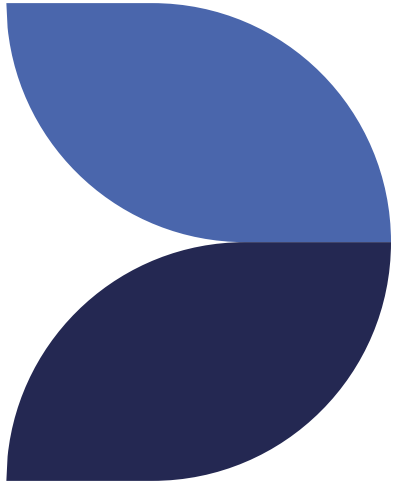
Is it possible to obtain the original source code by decompiling a binary?

- “Decompiler” takes an executable and de-compiles the instructions to source code
- No. Compilation is a lossy process. Decompilation is possible but a lot of context for human understanding, dead code, comments and generality will be lost permanently.
- Lots of information that isn’t needed by the executable program is lost. A simple example is that these two statements will produce the same code:

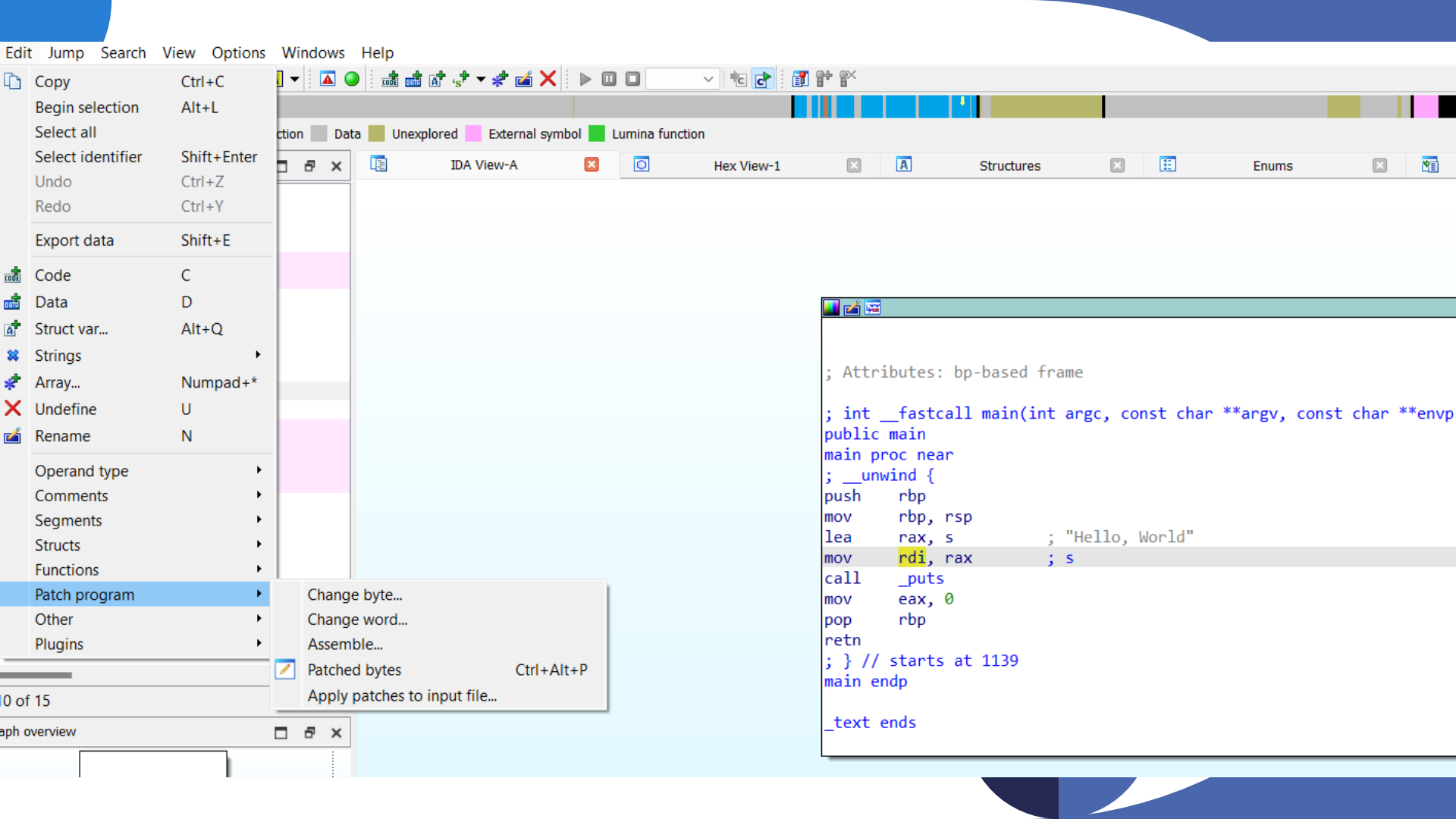
  a = b + c;

  x = y + z;
- Yes, the addresses will be different, but the actual instructions will be the same.
- Comments are also not part of the object code, nor is anything that can be optimized out. Loops sometimes get unrolled. Statements may get reordered.

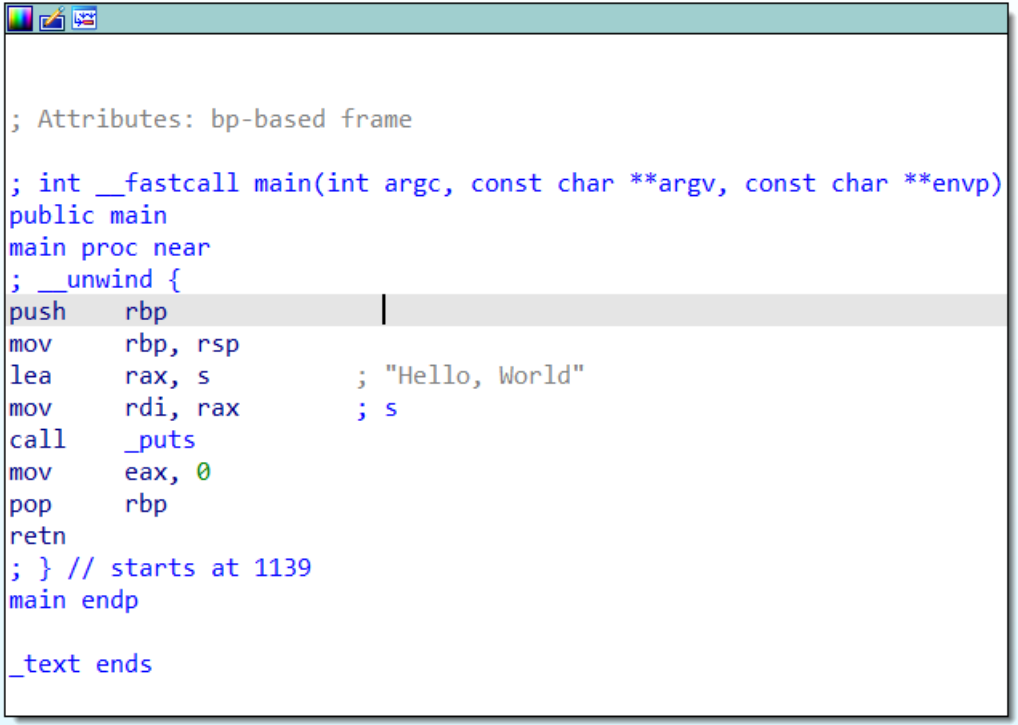
Binary Patching with IDA



- Patching, in this context, is a process of modifying the bytes or instructions of an executable program (ELF, EXE, etc.). Simply put, Patching allows us to make changes to the values or the flow of a program.


Edit Jump Search View Options Windows Help
Copy Ctrl+C
Begin selection Alt+L
Select all
Select identifier Shift+Enter
Undo Ctrl+Z
Redo Ctrl+Y
Export data Shift+E
Code C
Data D
Struct var... Alt+Q
Strings
Array... Numpad+*
Undefine U
Rename N
Operand type
Comments
Segments
Structs
Functions
Patch program
Other
Plugins
Change byte...
Change word...
Assemble...
Patched bytes Ctrl+Alt+P
Apply patches to input file...
Data Unexplored External symbol Lumina function
IDA View-A
Hex View-1
Structures
Enums
; Attributes: bp-based frame
; int __fastcall main(int argc, const char **argv, const char **envp
public main
main proc near
; __unwind {
push rbp
mov rbp, rsp
lea rax, s ; "Hello, World"
mov rdi, rax ; s
call _puts
mov eax, 0
pop rbp
retn
; } // starts at 1139
main endp
_text ends

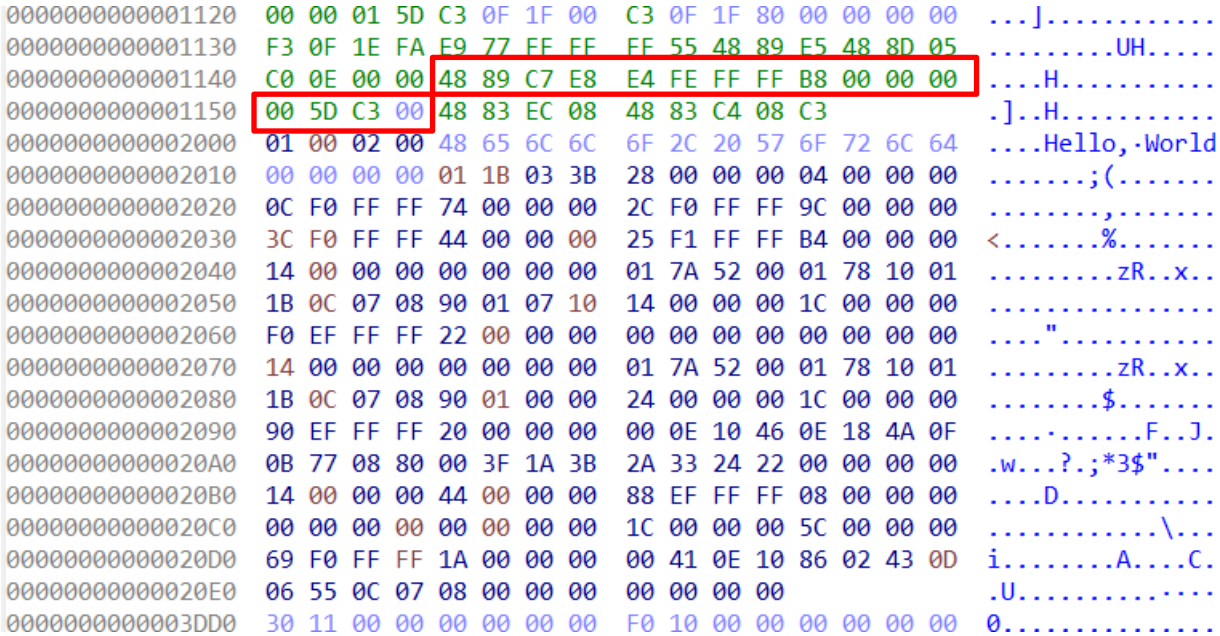
Double click **s**



```
; Attributes: bp-based frame

; int __fastcall main(int argc, const char **argv, const char **envp)
public main
main proc near
; __unwind {
push    rbp
mov     rbp, rsp
lea     rax, s          ; "Hello, World"
mov     rdi, rax        ; s
call    _puts
mov     eax, 0
pop     rbp
retn
; } // starts at 1139
main endp

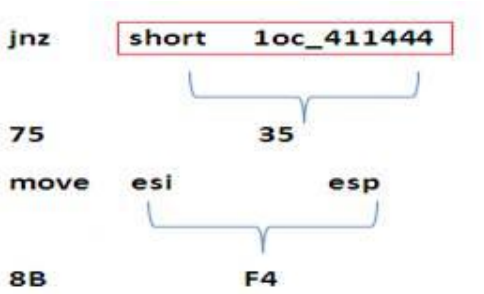
_text ends
```



```
0000000000001120  00 00 01 5D C3 0F 1F 00  C3 0F 1F 80 00 00 00 00  ...]............
0000000000001130  F3 0F 1E FA E9 77 FF FF  FF 55 48 89 E5 48 8D 05  .....w...UH.....
0000000000001140  C0 0E 00 00 48 89 C7 E8  E4 FE FF FF B8 00 00 00  ....H...........
0000000000001150  00 5D C3 00 48 83 EC 08  48 83 C4 08 C3           .]..H.....H....
0000000000002000  01 00 02 00 48 65 6C 6C  6F 2C 20 57 6F 72 6C 64  ....Hello, World
0000000000002010  00 00 00 00 01 1B 03 3B  28 00 00 00 04 00 00 00  .......;(.......
0000000000002020  0C F0 FF FF 74 00 00 00  2C F0 FF FF 9C 00 00 00  ....t...,.......
0000000000002030  3C F0 FF FF 44 00 00 00  25 F1 FF FF B4 00 00 00  <...D...%.......
0000000000002040  14 00 00 00 00 00 00 00  01 7A 52 00 01 78 10 01  .........zR..x..
0000000000002050  1B 0C 07 08 90 01 07 10  14 00 00 00 1C 00 00 00  ................
0000000000002060  F0 EF FF FF 22 00 00 00  00 00 00 00 00 00 00 00  ...."......"....
0000000000002070  14 00 00 00 00 00 00 00  01 7A 52 00 01 78 10 01  .........zR..x..
0000000000002080  1B 0C 07 08 90 01 00 00  24 00 00 00 1C 00 00 00  ........$.......
0000000000002090  90 EF FF FF 20 00 00 00  00 0E 10 46 0E 18 4A 0F  .... ......F..J.
00000000000020A0  0B 77 08 80 00 3F 1A 3B  2A 33 24 22 00 00 00 00  .w...?.;*3$"....
00000000000020B0  14 00 00 00 44 00 00 00  88 EF FF FF 08 00 00 00  ....D...........
00000000000020C0  00 00 00 00 00 00 00 00  1C 00 00 00 5C 00 00 00  ............\...
00000000000020D0  69 F0 FF FF 1A 00 00 00  00 41 0E 10 86 02 43 0D  i........A....C.
00000000000020E0  06 55 0C 07 08 00 00 00  00 00 00 00              .U..........
0000000000003DD0  30 11 00 00 00 00 00 00  F0 10 00 00 00 00 00 00  0...............
```

48 89 C7 E8 E4 FE FF FF B8 00 00 00 00 5D C3 00

The hex view offers a 16 bytes of sequence in one line and each two bytes represent only one assembly instruction set

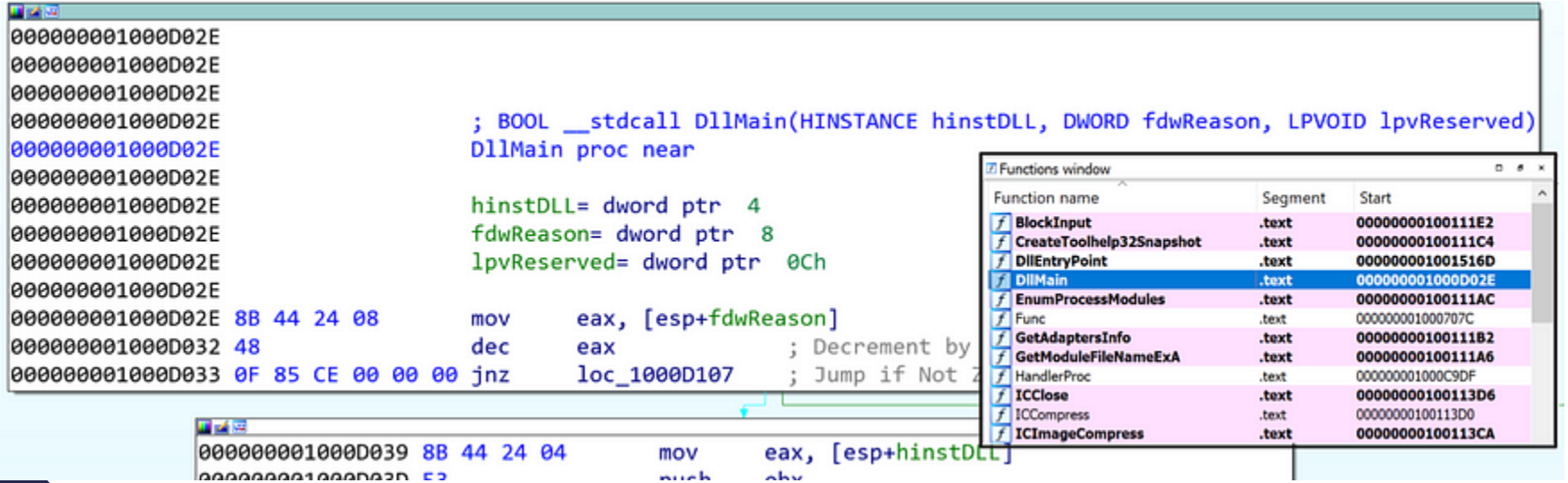



```
.text:00411408                  add     esp, 8
.text:0041140B                  test    eax, eax
.text:0041140D                  jnz     short loc_411444
.text:0041140F                  mov     esi, esp
.text:00411411                  push    offset aCongratulati
.text:00411416                  call    ds:printf
.text:0041141C                  add     esp, 4
.text:0041141F                  cmp     esi, esp
.text:00411421                  call    sub_411145
.text:00411426                  mov     esi, esp
.text:00411428                  push    offset aAjay      ; "a
.text:0041142D                  push    offset aReadyToLogir
.text:00411432                  call    ds:printf
.text:00411438                  add     esp, 8
.text:0041143B                  cmp     esi, esp
.text:0041143D                  call    sub_411145
.text:00411442                  jmp     short loc_41145B
.text:00411444 ; ---------------------------------------------
.text:00411444
.text:00411444 loc_411444:                              ; CO
.text:00411444                  mov     esi, esp
.text:00411446                  push    offset aWrongPasswor
```

```
004113CD  41 00 FF 15 C0 82 41 00  83 C4 04 3B F4 E8 66 FD  A. .+éA.â-.;(Ff²
004113DD  FF FF 8B F4 8D 45 94 50  68 8C 57 41 00 FF 15 C4    ï(ìEöPhîWA. .-
004113ED  82 41 00 83 C4 08 3B F4  E8 4B FD FF FF 68 84 57  éA.â-.;(FK²  häW
004113FD  41 00 8D 45 94 50 E8 9D  FC FF FF 83 C4 08 85 C0  A.ìEöPF¥n  â-.à+
0041140D  75 35 8B F4 68 6C 57 41  00 FF 15 C0 82 41 00 83  u5ï(hlWA. .+éA.â
0041141D  C4 04 3B F4 E8 1F FD FF  FF 8B F4 68 84 57 41 00  -.;(F.²  ï(häWA.
0041142D  68 50 57 41 00 FF 15 C0  82 41 00 83 C4 08 3B F4  hPWA. .+éA.â-.;(
0041143D  E8 03 FD FF FF EB 17 8B  F4 68 3C 57 41 00 FF 15  F.²  d.ï(h<WA. .
0041144D  C0 82 41 00 83 C4 04 3B  F4 E8 EA FC FF FF 8B F4  +éA.â-.;(FOn  ï(
```

What is the address of DllMain?

The address off DllMain is 0x1000D02E. This can be found within the graph mode, or within the Functions window



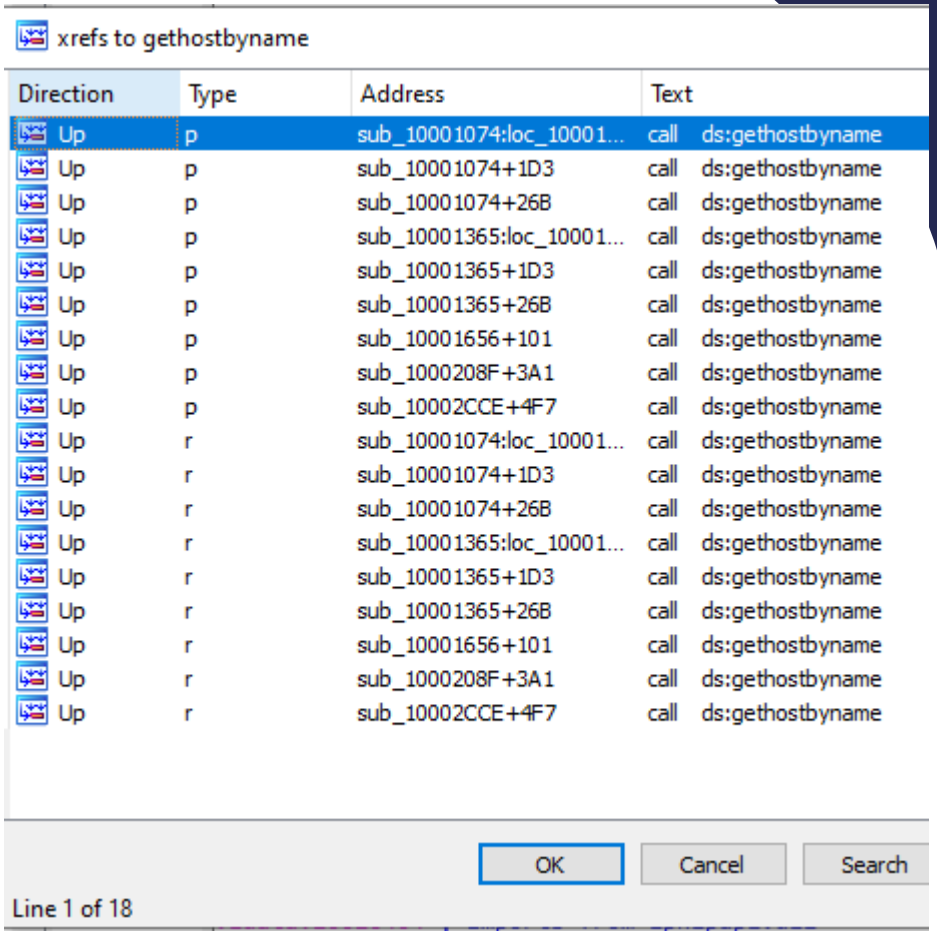
Where is the import gethostbyname located?

gethostbyname is located at 0x100163CC. This is found through the Imports window and double-clicking the function.

```
.idata:100163CC ; struct hostent *__stdcall gethostbyname(const char *name)
.idata:100163CC                 extrn gethostbyname:dword
.idata:100163CC                                         ; CODE XREF: sub_10001074:loc_100011AF↑p
.idata:100163CC                                         ; sub_10001074+1D3↑p ...
```

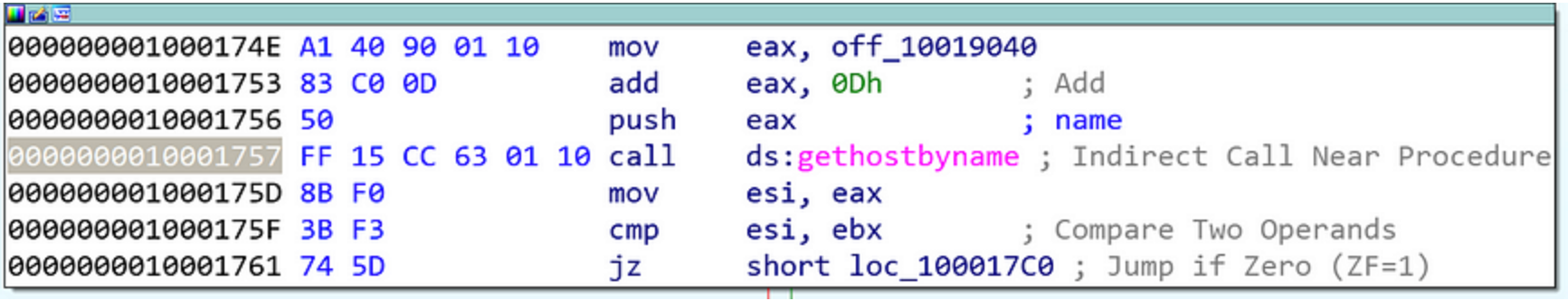
How many functions call gethostbyname?

Searching the xrefs (ctrl+x) on gethostbyname shows it is referenced 18 times, 9 of which are type (p) for the near calll, and the other 9 are read (r)



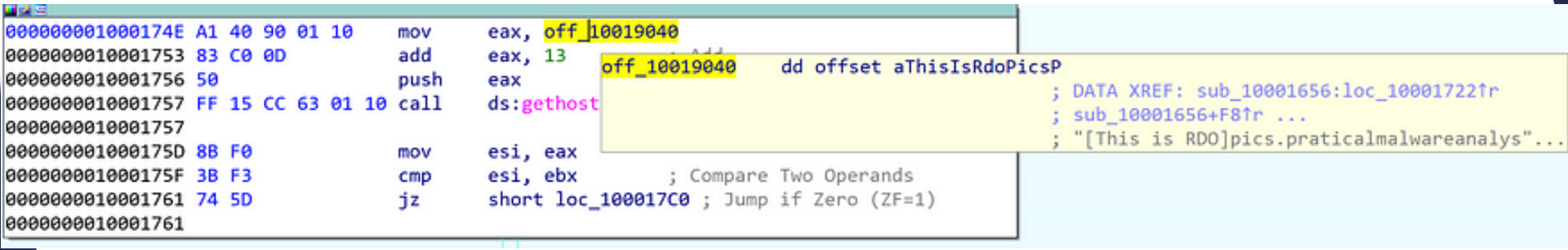
For gethostbyname at 0x10001757, which DNS request is made?

call to thegethostbyname function, which we know takes one parameter; in this case, whatever is in eax — the contents of off_10019040



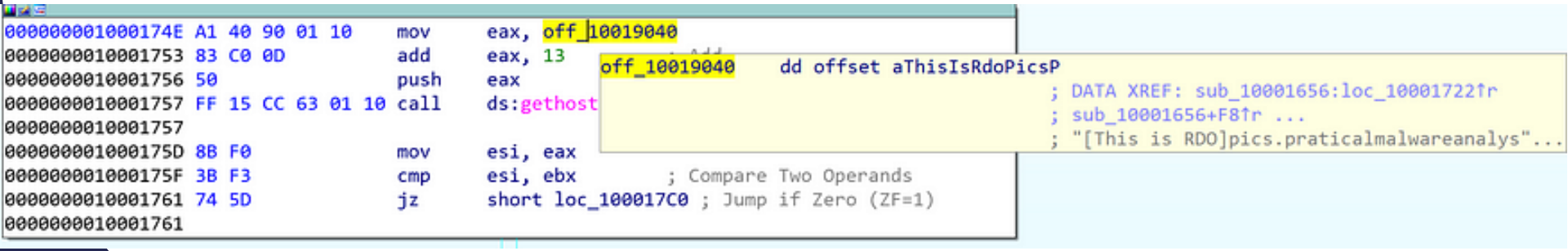
For gethostbyname at 0x10001757, which DNS request is made?

The contents of off_10019040 points to a variable aThisIsRdoPicsP which contains the string [This is RDO]pics.practicalmalwareanalysis.com. This is moved into eax



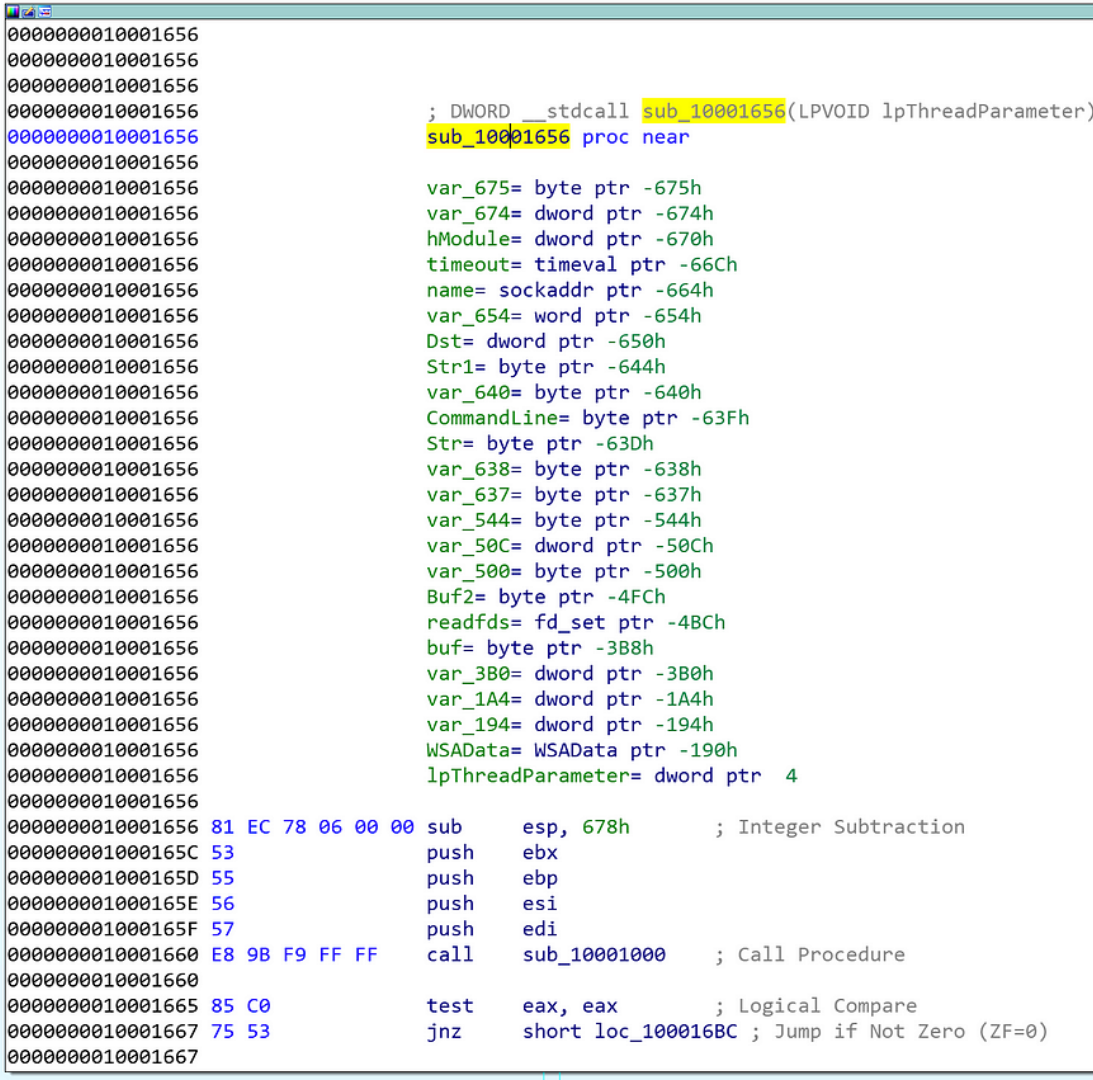
For gethostbyname at 0x10001757, which DNS request is made?

- Importantly, 0Dh is added to eax, which moves the pointer along the current contents. 0Dh can be converted in IDA by pressing H, to 13.
- This means the eax now points to 13 characters inside of its current contents, skipping past the prefix [This is RDO] and resulting in the DNS request being made for **pics.practicalmalwareanalysis.com**.



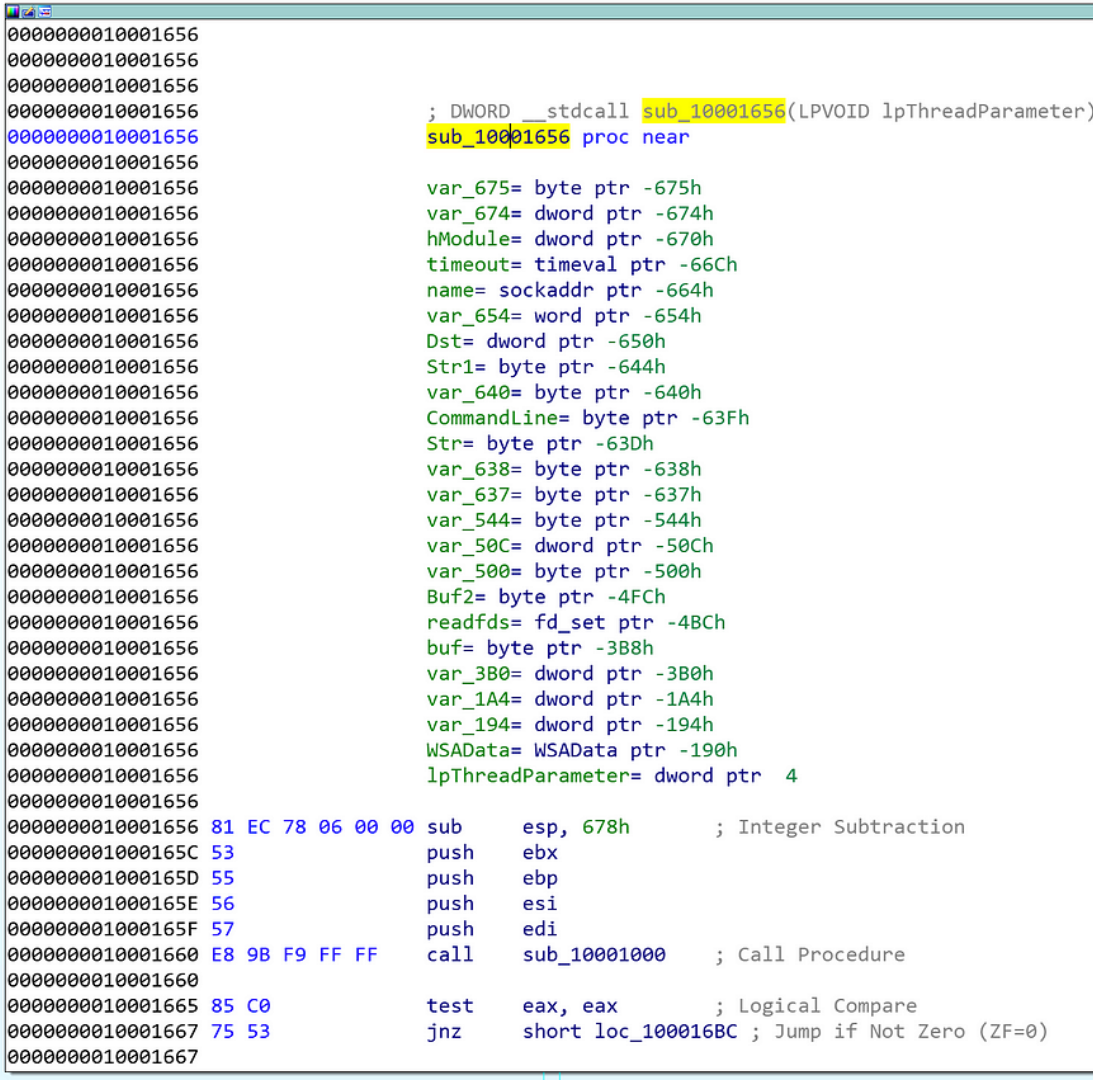
How many parameters and local variables are recognized for the subroutine at 0x10001656?

There are a total of 24 variables and parameters for sub_10001656



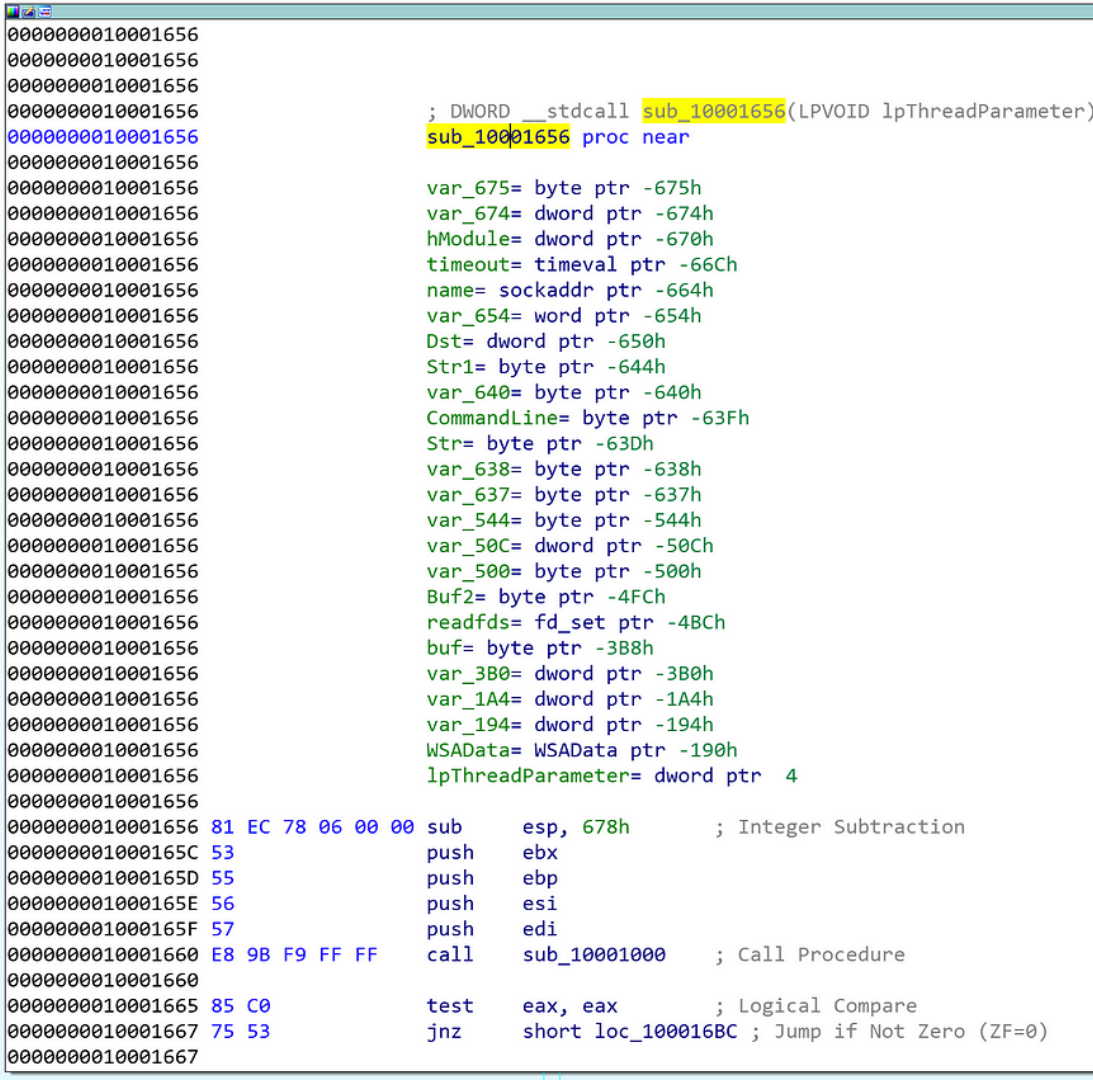
```
0000000010001656
0000000010001656
0000000010001656
0000000010001656                    ; DWORD __stdcall sub_10001656(LPVOID lpThreadParameter)
0000000010001656                    sub_10001656 proc near
0000000010001656
0000000010001656                    var_675= byte ptr -675h
0000000010001656                    var_674= dword ptr -674h
0000000010001656                    hModule= dword ptr -670h
0000000010001656                    timeout= timeval ptr -66Ch
0000000010001656                    name= sockaddr ptr -664h
0000000010001656                    var_654= word ptr -654h
0000000010001656                    Dst= dword ptr -650h
0000000010001656                    Str1= byte ptr -644h
0000000010001656                    var_640= byte ptr -640h
0000000010001656                    CommandLine= byte ptr -63Fh
0000000010001656                    Str= byte ptr -63Dh
0000000010001656                    var_638= byte ptr -638h
0000000010001656                    var_637= byte ptr -637h
0000000010001656                    var_544= byte ptr -544h
0000000010001656                    var_50C= dword ptr -50Ch
0000000010001656                    var_500= byte ptr -500h
0000000010001656                    Buf2= byte ptr -4FCh
0000000010001656                    readfds= fd_set ptr -4BCh
0000000010001656                    buf= byte ptr -3B8h
0000000010001656                    var_3B0= dword ptr -3B0h
0000000010001656                    var_1A4= dword ptr -1A4h
0000000010001656                    var_194= dword ptr -194h
0000000010001656                    WSAData= WSAData ptr -190h
0000000010001656                    lpThreadParameter= dword ptr  4
0000000010001656
0000000010001656 81 EC 78 06 00 00  sub     esp, 678h       ; Integer Subtraction
000000001000165C 53                 push    ebx
000000001000165D 55                 push    ebp
000000001000165E 56                 push    esi
000000001000165F 57                 push    edi
0000000010001660 E8 9B F9 FF FF     call    sub_10001000    ; Call Procedure
0000000010001660
0000000010001665 85 C0              test    eax, eax        ; Logical Compare
0000000010001667 75 53              jnz     short loc_100016BC ; Jump if Not Zero (ZF=0)
0000000010001667
```

Local variables correspond to negative offsets, where there are 23. Many are generated by IDA and prepended with var_ however there are some which have been resolved, such as name or commandline



```
0000000010001656
0000000010001656
0000000010001656
0000000010001656                    ; DWORD __stdcall sub_10001656(LPVOID lpThreadParameter)
0000000010001656                    sub_10001656 proc near
0000000010001656
0000000010001656                    var_675= byte ptr -675h
0000000010001656                    var_674= dword ptr -674h
0000000010001656                    hModule= dword ptr -670h
0000000010001656                    timeout= timeval ptr -66Ch
0000000010001656                    name= sockaddr ptr -664h
0000000010001656                    var_654= word ptr -654h
0000000010001656                    Dst= dword ptr -650h
0000000010001656                    Str1= byte ptr -644h
0000000010001656                    var_640= byte ptr -640h
0000000010001656                    CommandLine= byte ptr -63Fh
0000000010001656                    Str= byte ptr -63Dh
0000000010001656                    var_638= byte ptr -638h
0000000010001656                    var_637= byte ptr -637h
0000000010001656                    var_544= byte ptr -544h
0000000010001656                    var_50C= dword ptr -50Ch
0000000010001656                    var_500= byte ptr -500h
0000000010001656                    Buf2= byte ptr -4FCh
0000000010001656                    readfds= fd_set ptr -4BCh
0000000010001656                    buf= byte ptr -3B8h
0000000010001656                    var_3B0= dword ptr -3B0h
0000000010001656                    var_1A4= dword ptr -1A4h
0000000010001656                    var_194= dword ptr -194h
0000000010001656                    WSAData= WSAData ptr -190h
0000000010001656                    lpThreadParameter= dword ptr  4
0000000010001656
0000000010001656 81 EC 78 06 00 00  sub     esp, 678h       ; Integer Subtraction
000000001000165C 53                 push    ebx
000000001000165D 55                 push    ebp
000000001000165E 56                 push    esi
000000001000165F 57                 push    edi
0000000010001660 E8 9B F9 FF FF     call    sub_10001000    ; Call Procedure
0000000010001660
0000000010001665 85 C0              test    eax, eax        ; Logical Compare
0000000010001667 75 53              jnz     short loc_100016BC ; Jump if Not Zero (ZF=0)
0000000010001667
```

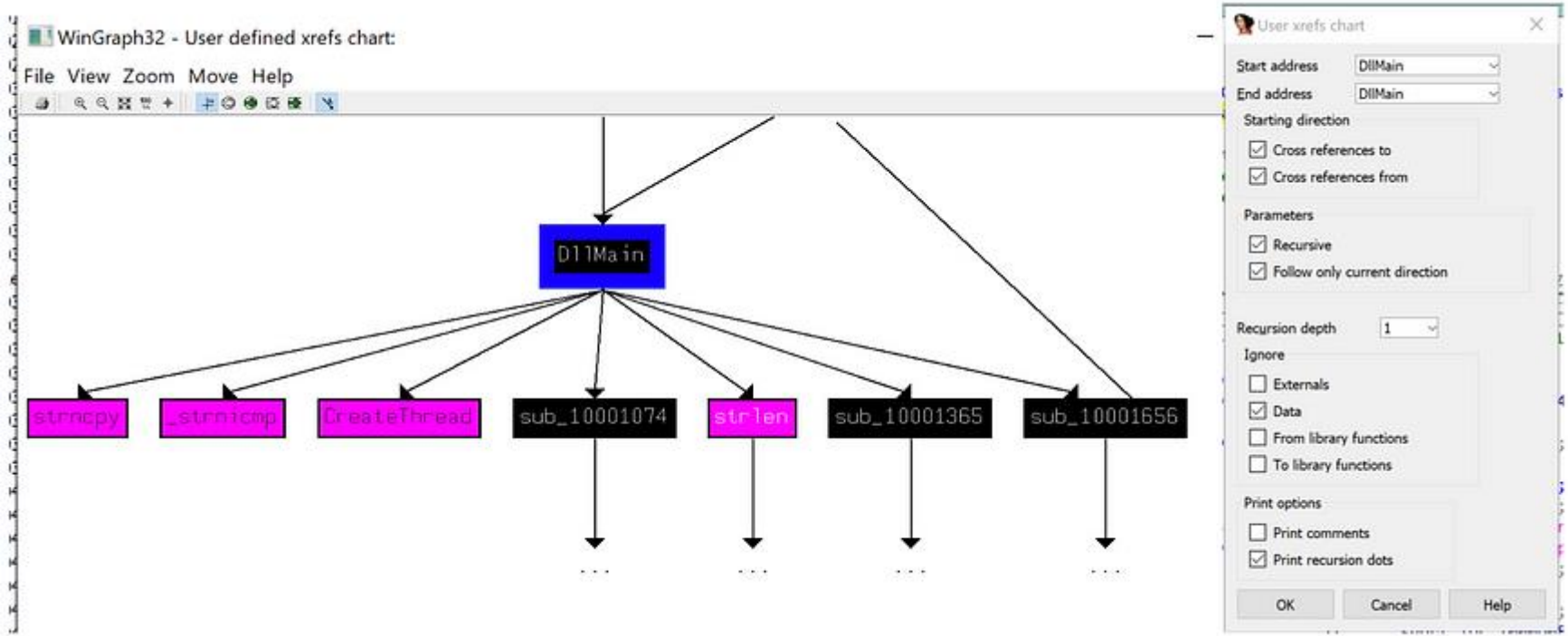
Parameters have positive offsets. Here there is one, currently lpThreadParameter. This may also be seen as arg_0 if not automagically resolved.



```
0000000010001656
0000000010001656
0000000010001656
0000000010001656                    ; DWORD __stdcall sub_10001656(LPVOID lpThreadParameter)
0000000010001656                    sub_10001656 proc near
0000000010001656
0000000010001656                    var_675= byte ptr -675h
0000000010001656                    var_674= dword ptr -674h
0000000010001656                    hModule= dword ptr -670h
0000000010001656                    timeout= timeval ptr -66Ch
0000000010001656                    name= sockaddr ptr -664h
0000000010001656                    var_654= word ptr -654h
0000000010001656                    Dst= dword ptr -650h
0000000010001656                    Str1= byte ptr -644h
0000000010001656                    var_640= byte ptr -640h
0000000010001656                    CommandLine= byte ptr -63Fh
0000000010001656                    Str= byte ptr -63Dh
0000000010001656                    var_638= byte ptr -638h
0000000010001656                    var_637= byte ptr -637h
0000000010001656                    var_544= byte ptr -544h
0000000010001656                    var_50C= dword ptr -50Ch
0000000010001656                    var_500= byte ptr -500h
0000000010001656                    Buf2= byte ptr -4FCh
0000000010001656                    readfds= fd_set ptr -4BCh
0000000010001656                    buf= byte ptr -3B8h
0000000010001656                    var_3B0= dword ptr -3B0h
0000000010001656                    var_1A4= dword ptr -1A4h
0000000010001656                    var_194= dword ptr -194h
0000000010001656                    WSAData= WSAData ptr -190h
0000000010001656                    lpThreadParameter= dword ptr  4
0000000010001656
0000000010001656 81 EC 78 06 00 00  sub     esp, 678h       ; Integer Subtraction
000000001000165C 53                 push    ebx
000000001000165D 55                 push    ebp
000000001000165E 56                 push    esi
000000001000165F 57                 push    edi
0000000010001660 E8 9B F9 FF FF     call    sub_10001000    ; Call Procedure
0000000010001660
0000000010001665 85 C0              test    eax, eax        ; Logical Compare
0000000010001667 75 53              jnz     short loc_100016BC ; Jump if Not Zero (ZF=0)
0000000010001667
```

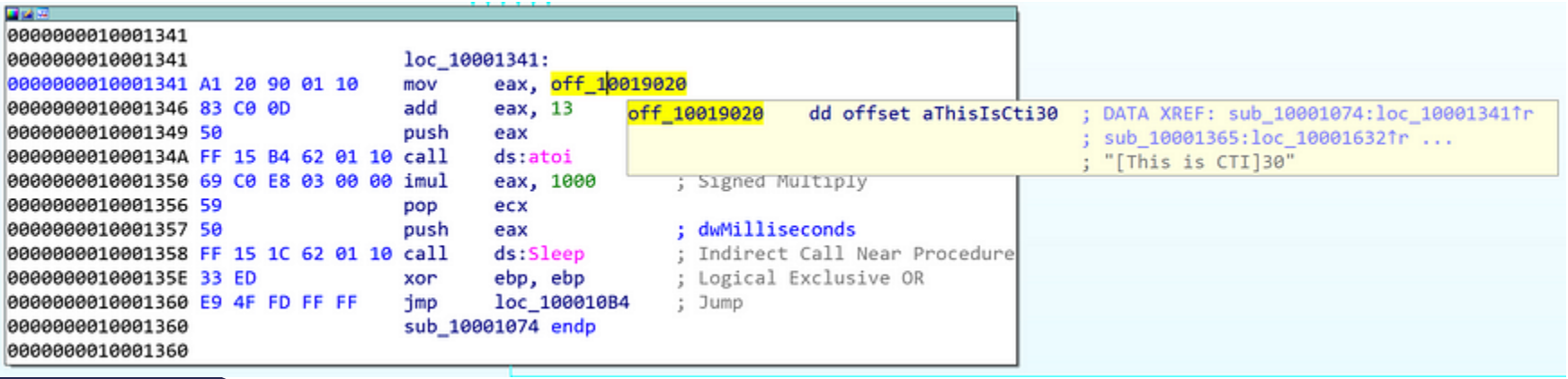
How many Windows API functions does DllMain call directly, and how many at a depth of 2?

View -> Graphs -> User XRef Chart. Set start and end addresses to DllMain and the Recursion depth to 2



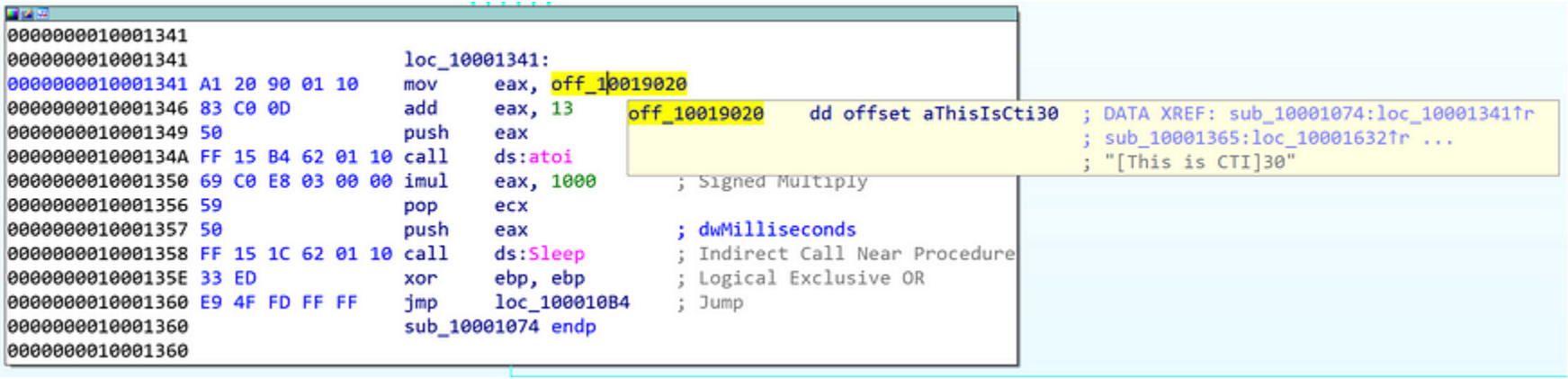
How long will the Sleep API function at 0x10001358 execute for?

- At first glance, one might think that the value passed to the sleep is **3E8h (1000),** equating to 1 second, however it is a imul call which means the value at eax is getting multiplied by 1000.
- Looking up, we see that **aThisIsCti30** at the offset address is moved into **eax** and then the pointer is moved 13 along



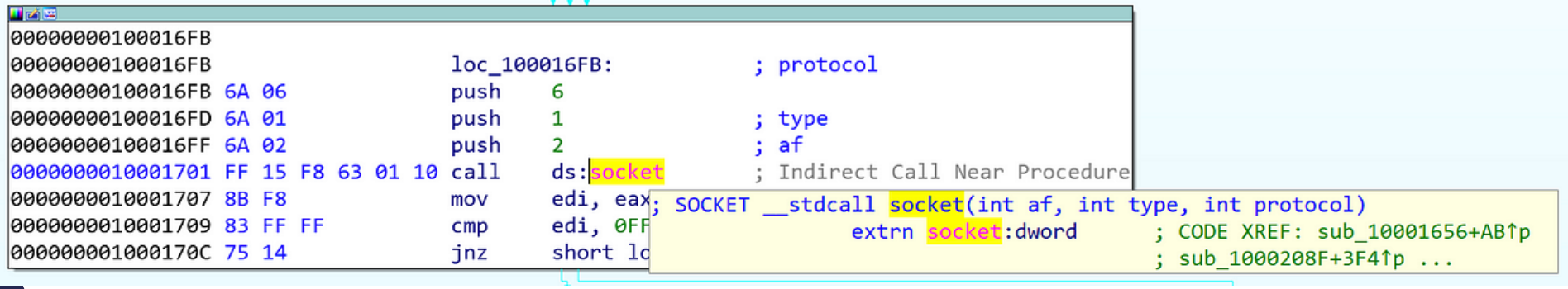
How long will the Sleep API function at 0x10001358 execute for?

- This means that the value of eax when it is pushed is 30. atoi converts the string to an integer, and it is multiplied by 1000.
- Therefore, the Sleep API function sleeps for 30 seconds.

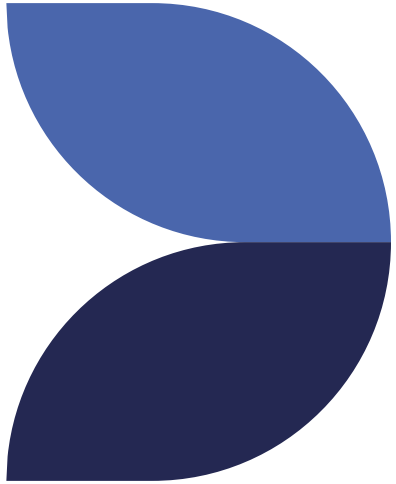


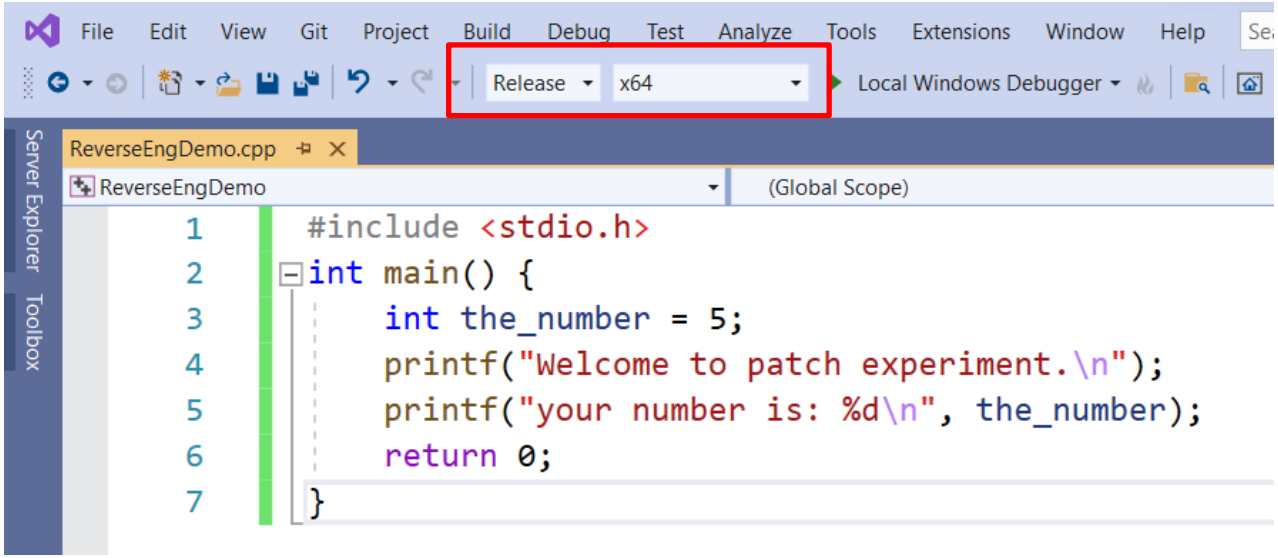
What are the three parameters for the call to socket at 0x10001701?

The three values pushed to the stack, labeled as protocol, type, and af, and are 6, 1, 2 respectively, are the three parameters used for the call to socket



BINARY PATCHING

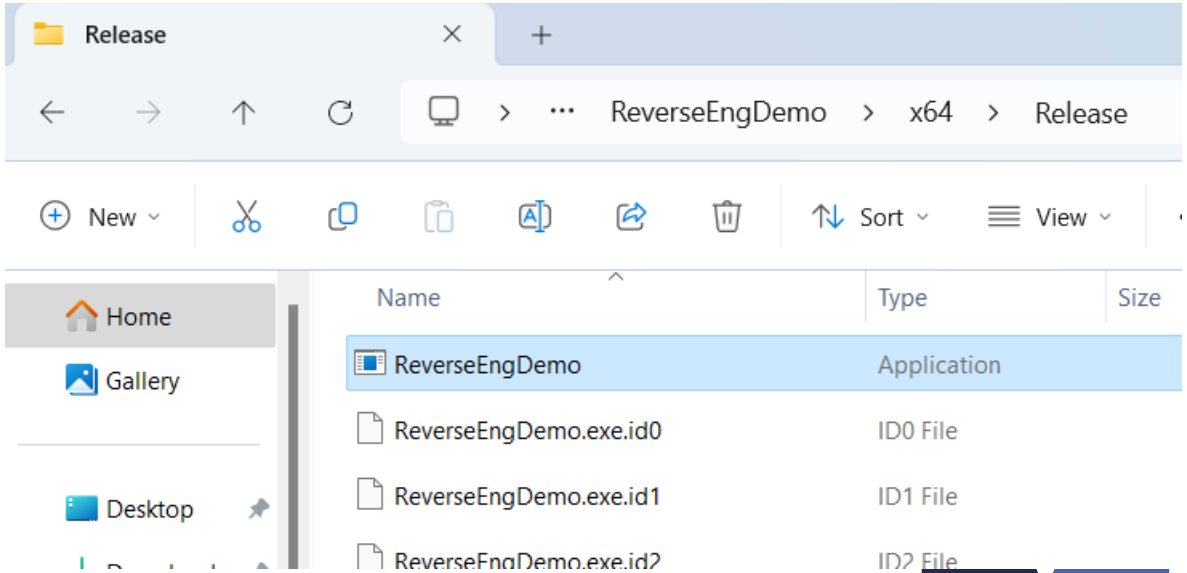


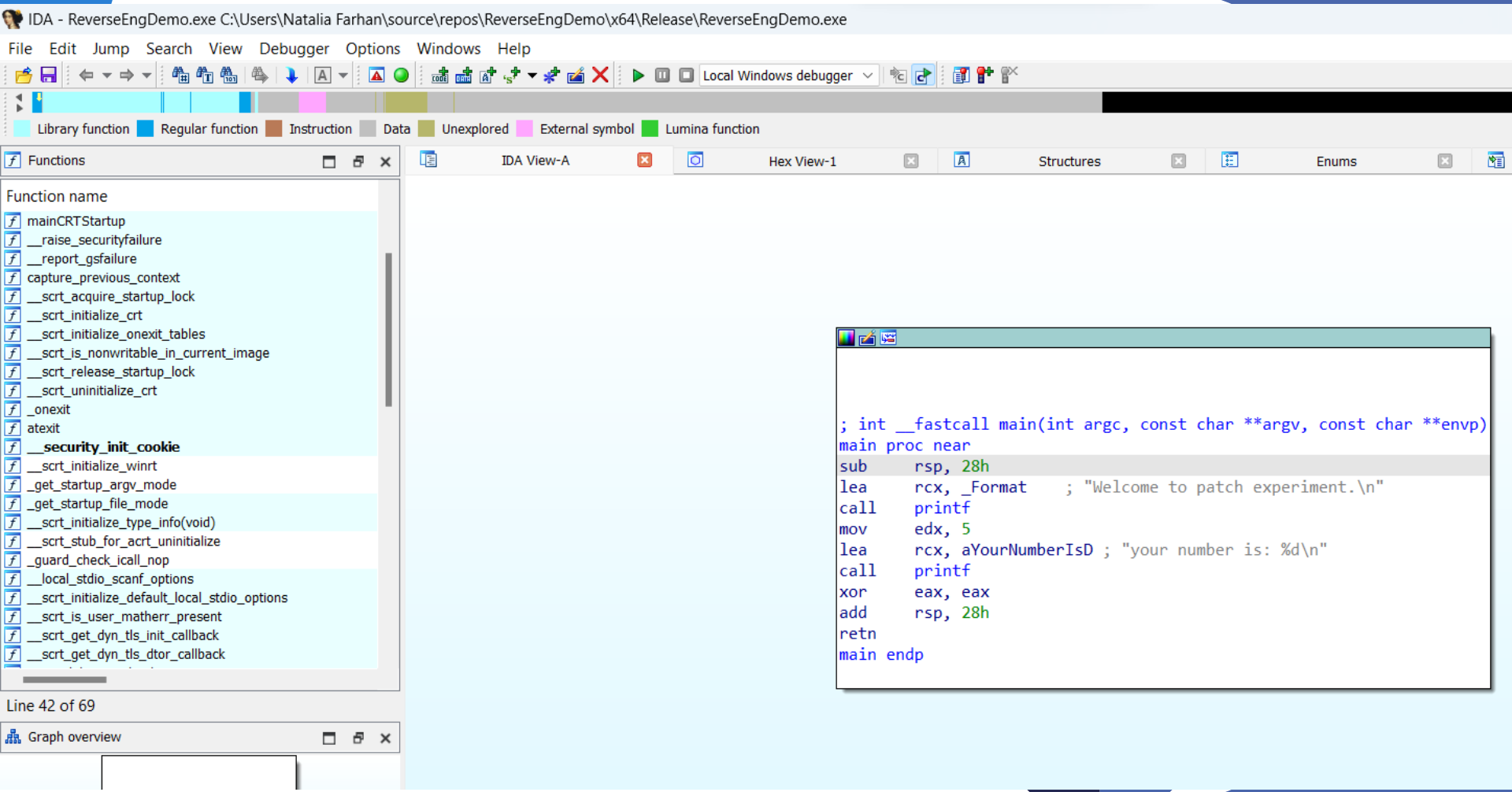



```
#include <stdio.h>
int main(){
 int the_number = 5;
 printf("Welcome to patch experiment.\n");
 printf("your number is: %d\n", the_number);
 return 0;
}
```

we want to try to change the “the_number” value from 5 to another value using the patching technique

- Drag exe file to IDA freeware icon




IDA - ReverseEngDemo.exe C:\Users\Natalia Farhan\source\repos\ReverseEngDemo\x64\Release\ReverseEngDemo.exe
File Edit Jump Search View Debugger Options Windows Help
Local Windows debugger
Library function Regular function Instruction Data Unexplored External symbol Lumina function
Functions
Function name
mainCRTStartup
__raise_securityfailure
__report_gsfailure
capture_previous_context
__scrt_acquire_startup_lock
__scrt_initialize_crt
__scrt_initialize_onexit_tables
__scrt_is_nonwritable_in_current_image
__scrt_release_startup_lock
__scrt_uninitialize_crt
_onexit
atexit
__security_init_cookie
__scrt_initialize_winrt
_get_startup_argv_mode
_get_startup_file_mode
__scrt_initialize_type_info(void)
__scrt_stub_for_acrt_uninitialize
_guard_check_icall_nop
__local_stdio_scanf_options
__scrt_initialize_default_local_stdio_options
__scrt_is_user_matherr_present
__scrt_get_dyn_tls_init_callback
__scrt_get_dyn_tls_dtor_callback
Line 42 of 69
Graph overview
IDA View-A
Hex View-1
Structures
Enums
; int __fastcall main(int argc, const char **argv, const char **envp)
main proc near
sub rsp, 28h
lea rcx, _Format ; "Welcome to patch experiment.\n"
call printf
mov edx, 5
lea rcx, aYourNumberIsD ; "your number is: %d\n"
call printf
xor eax, eax
add rsp, 28h
retn
main endp

pseudocode view (hit F5 on keyboard)

IDA View-A | Pseudocode-A | Hex View-1

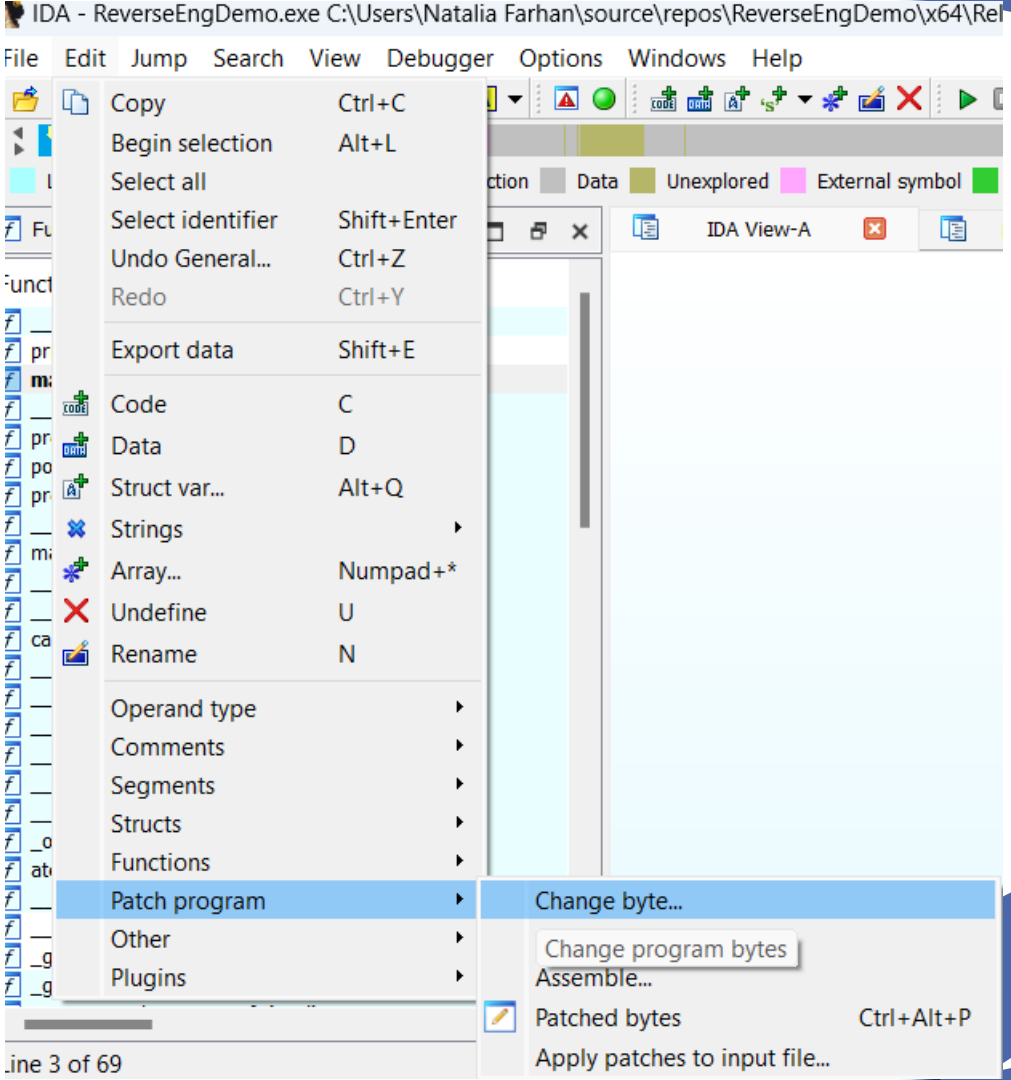
```
1 int __fastcall main(int argc, const char **argv, const char **envp)
2 {
3   printf("Welcome to patch experiment.\n");
4   printf("your number is: %d\n", 5i64);
5   return 0;
6 }
```

```
; int __fastcall main(int argc, const char **argv, const char **envp)
main proc near
sub     rsp, 28h
lea     rcx, _Format    ; "Welcome to patch experiment.\n"
call    printf
mov     edx, 5
lea     rcx, aYourNumberIsD ; "your number is: %d\n"
call    printf
xor     eax, eax
add     rsp, 28h
retn
main endp
```

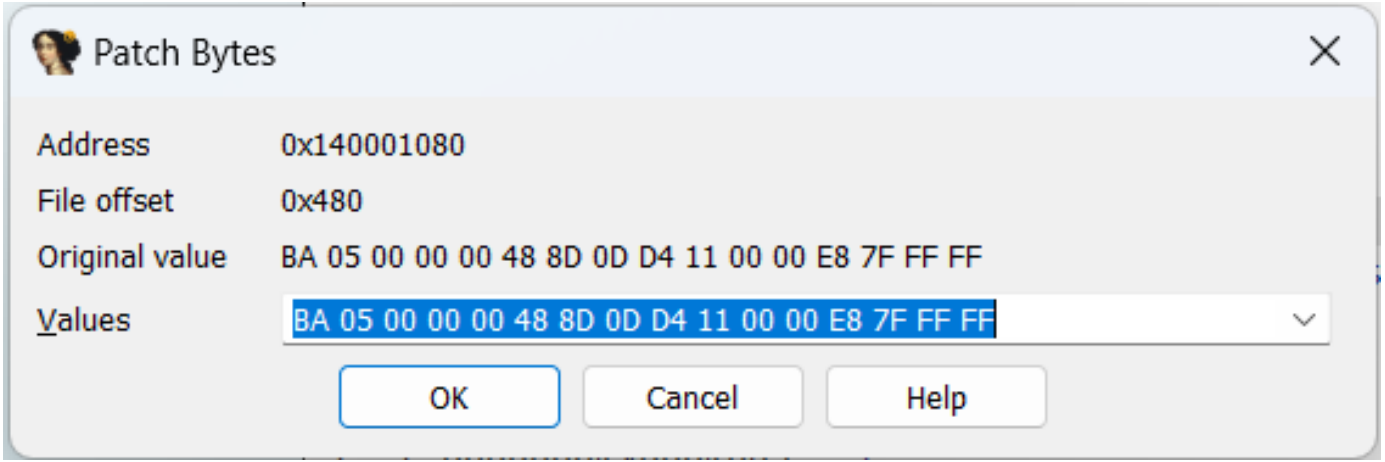
.text:0000000140001080 mov edx, 5

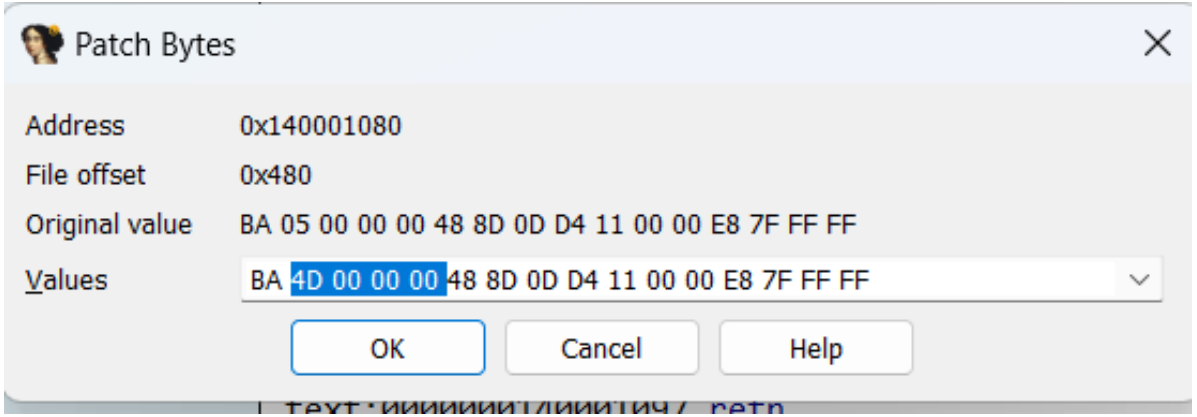
- integer 5, which is the same value as “the_number” variable in our source code. Indicating that the value 5 is stored in program [rbp+var_4] register through the mov instruction.

- To change the value, just click and block the integer 5, then go to the **Edit tab->Patch Program->Change bytes.**


IDA - ReverseEngDemo.exe C:\Users\Natalia Farhan\source\repos\ReverseEngDemo\x64\Rel
File
Edit
Jump
Search
View
Debugger
Options
Windows
Help
Copy Ctrl+C
Begin selection Alt+L
Select all
Select identifier Shift+Enter
Undo General... Ctrl+Z
Redo Ctrl+Y
Export data Shift+E
Code C
Data D
Struct var... Alt+Q
Strings
Array... Numpad+*
Undefine U
Rename N
Operand type
Comments
Segments
Structs
Functions
Patch program
Other
Plugins
Change byte...
Change program bytes
Assemble...
Patched bytes Ctrl+Alt+P
Apply patches to input file...
Data
Unexplored
External symbol
IDA View-A
ine 3 of 69

- The value will be displayed in Hexadecimal form, and we can see the value 05 00 00 00 where 05 is the hex value of integer 5.
- Suppose we want to change the value to 77 which has a hex value of 4d, we just need to change the 05 00 00 00 part to 4D 00 00 00. And then hit “OK”.



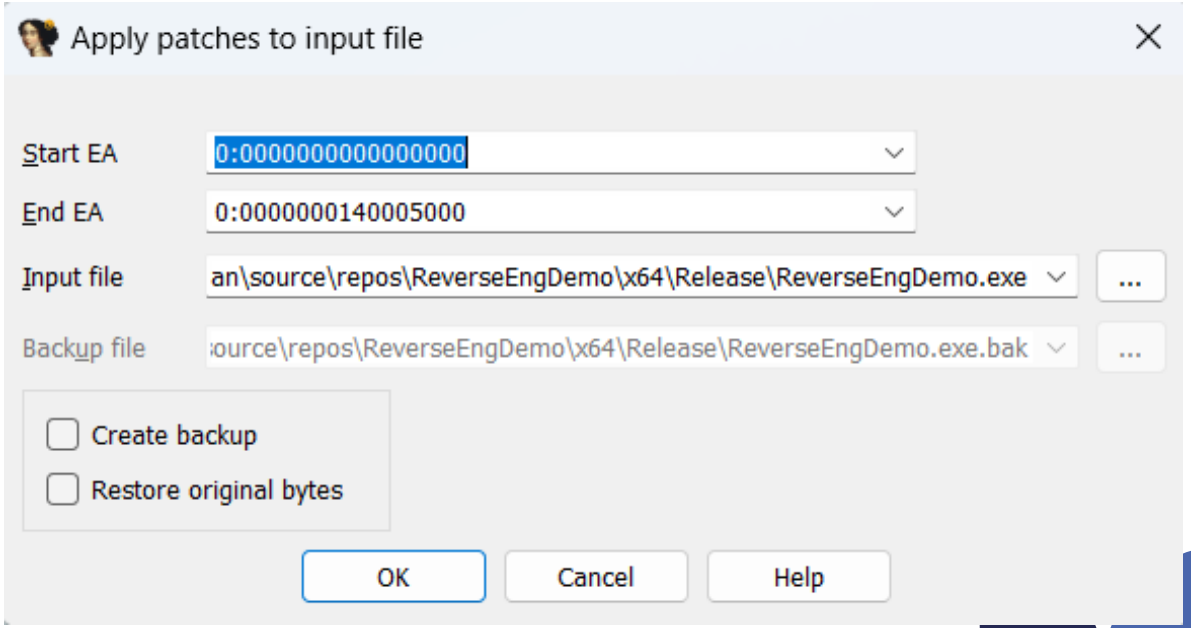

Patch Bytes
Address
0x140001080
File offset
0x480
Original value
BA 05 00 00 00 48 8D 0D D4 11 00 00 E8 7F FF FF
Values
BA 4D 00 00 00 48 8D 0D D4 11 00 00 E8 7F FF FF
OK
Cancel
Help

H - toggle between hexadecimal and decimal representations.

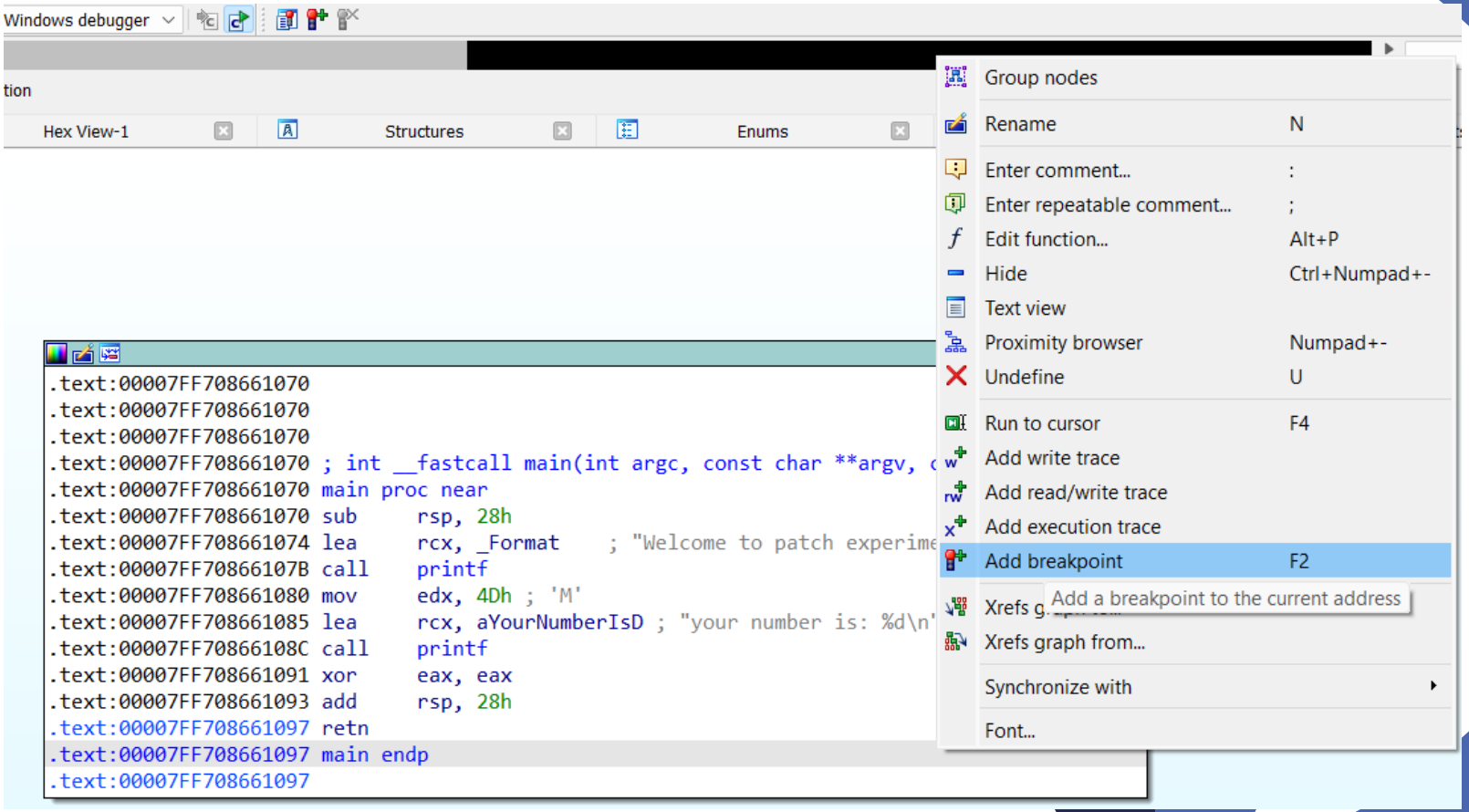
```
.text:0000000140001070
.text:0000000140001070
.text:0000000140001070
.text:0000000140001070 ; int __fastcall main(int argc, const char **argv, const char **envp)
.text:0000000140001070 main proc near
.text:0000000140001070 sub     rsp, 28h
.text:0000000140001074 lea     rcx, _Format    ; "Welcome to patch experiment.\n"
.text:000000014000107B call    printf
.text:0000000140001080 mov     edx, 77
.text:0000000140001085 lea     r
.text:000000014000108C call    p
.text:0000000140001091 xor     e
.text:0000000140001093 add     r
.text:0000000140001097 retn
.text:0000000140001097 main endp
.text:0000000140001097
```

```
.text:0000000140001070
.text:0000000140001070
.text:0000000140001070
.text:0000000140001070 ; int __fastcall main(int argc, const char **argv, const char **envp)
.text:0000000140001070 main proc near
.text:0000000140001070 sub     rsp, 28h
.text:0000000140001074 lea     rcx, _Format    ; "Welcome to patch experiment.\n"
.text:000000014000107B call    printf
.text:0000000140001080 mov     edx, 4Dh ; 'M'
.text:0000000140001085 lea     rcx, aYourNumberIsD ; "your number is: %d\n"
.text:000000014000108C call    printf
.text:0000000140001091 xor     eax, eax
.text:0000000140001093 add     rsp, 28h
.text:0000000140001097 retn
.text:0000000140001097 main endp
.text:0000000140001097
```

next step is we recompile the program again into ELF with the changed bytes. You do this by clicking the **Edit tab->Patch program->Apply patches to input file**.



run the patched program (add breakpoint to make the screen stay)




C:\Users\Natalia Farhan\sourc
Welcome to patch experiment.
your number is: 77